1. ML-guided ASIC technology mapping

Design an ML-guided cut-selection for ASIC technology mapping using ABC. For a design, you need to choose good cuts to reduce the number of cuts used for each node during mapping.

ToDo:

1. Show how the mapping time (CPU time) varies with the number of cuts used in the mapper.

2. Describe your ML-model

3. Compare the area-delay QoR with ABC-TM

Note: You can dump all cuts into a file and then select good cuts for each node. However, reading and writing cuts from and to a file slows down the TM process. Ideally you should change the ABC codebase and filter cuts directly.

2. Power-aware recipe synthesis

Given an AIG, find a recipe that minimizes power by using a power predictor.

ToDo:

1. Specify how you have designed the power predictor. How is it different from area and/or delay predictor?

2. Show the performance of your power predictor on unseen designs

3. Compare the power value of the recipe obtained by your method with the recipes available in abc.rc file. You can run the recipes available in abc.rc multiple times within the time budget to have a fair comparison with your method.

3. Macro Placement Optimization using ISPD 2005 Benchmarks and OpenROAD

1. Introduction

This project focuses on improving **macro placement** using the **ISPD 2005 placement benchmark dataset**. An optimization model (e.g., Simulated Annealing, ML, RL, or other heuristics) will generate an **updated placement file (.pl)** with improved macro positions.

The updated Bookshelf design will then be converted to **LEF/DEF using RosettaStone** and loaded into **OpenROAD** for **standard cell placement**. Finally, the resulting placement will be evaluated using metrics such as **wirelength and congestion** and compared with the **baseline design**.

2. Dataset: ISPD 2005 Placement Benchmarks

The ISPD 2005 benchmarks are widely used in placement research. These benchmarks use the Bookshelf file format, which describes circuit components, connectivity, and placement information.

Dataset link:
https://www.ispd.cc/contests/05/contest.htm

Each benchmark design (for example: adaptec1, adaptec2, etc.) contains several Bookshelf files that together describe the circuit.

3. Bookshelf File Format

Each design in the dataset contains several important files that describe different aspects of the circuit design.

3.1 .nodes File

This file defines every physical object (node) in the circuit and its dimensions (width and height).
It distinguishes between movable components (like standard cells) and fixed components (like I/O pads or hard macros).

Format: NodeName Width Height [terminal]

If a node has the terminal or Fixed keyword, it means its location is fixed and cannot be moved by the placement algorithm.

```
NumNodes : 4
NumTerminals : 1

NodeA  10  20
NodeB  15  20
NodeC  20  40
Pad1   10  10  terminal
```

3.2 .nets File

This is the most critical file for wirelength optimization. It defines the netlist—the topological connectivity of the circuit.
It lists every wire (net) and specifies which pins on which nodes are connected by that wire.

It first declares the "degree" of the net (how many pins are connected to it).
Then, it lists the specific nodes attached to that net, along with the physical offset (x, y) of the pin relative to the center of the node.

```
NumNets : 2
NumPins : 5

NetDegree : 3
NodeA I : 0.0 0.0
NodeB O : 2.5 -1.0
Pad1  I : 0.0 0.0

NetDegree : 2
NodeB I : -2.5 1.0
NodeC O : 0.0 0.0
```

(In this snippet, I and O signify input/output pins, and the numbers are the x/y offsets from the node's origin).

3.3 .pl File (Placement File)

This file records the exact (x, y) coordinates of every node.

Initial .pl file:
When you download a benchmark, the provided .pl file usually only contains the coordinates of the fixed terminals
(I/O pads and fixed macros). The standard cells are all set to (0,0)

Output .pl file:
After an algorithm finishes running, it outputs a new .pl file containing the optimized coordinates for all the movable nodes.
Evaluators then use this output file, alongside the .nets file, to calculate the final wirelength.

```
NodeA  150  100  : N
NodeB  180  100  : FS
NodeC  200  120  : N
Pad1   0    0    : N /FIXED
```

(The N, FS, etc., denote the orientation of the cell, such as North, Flipped-South, etc.)

3.4 .scl File

This file defines the physical "canvas" of the chip. It specifies the core placement area by defining the layout rows
where standard cells can be legally placed.

It defines the starting coordinate of the row, its length, its height, and the "site width"
(the minimum legal unit of horizontal placement).

It also defines the orientation of the row (e.g., whether cells placed in this row should be flipped).

```
NumRows : 2

CoreRow Horizontal
  Coordinate   : 100
  Height       : 20
  Sitewidth    : 1
  Sitespacing  : 1
  Siteorient   : N
  Sitesymmetry : Y
  NumSites     : 500
End
```

3.5 .aux File

The .aux file is a reference file that lists the other Bookshelf files belonging to the design.

Example:
RowBasedPlacement : adaptec1.nodes adaptec1.nets adaptec1.pl adaptec1.scl

4. Preparing Input for the Optimization Model

The goal of the model is to optimize macro placement.

Information extracted from the dataset includes:
- Macro names and sizes from the .nodes file
- Connectivity information from the .nets file
- Initial placement from the .pl file
- Chip layout constraints from the .scl file

5. Optimization Model

Students can design any optimization algorithm to improve macro placement such as simulated annealing, reinforcement learning, neural networks, or heuristic approaches.

The model should output new macro coordinates which will be written into an updated .pl file.

Example output:

U1 150.0 220.0
U2 420.0 180.0

6. Converting Bookshelf Format to LEF/DEF

Modern physical design tools do not directly use the Bookshelf format. Therefore the Bookshelf files must be converted to LEF and DEF formats.

RosettaStone repository:
https://github.com/ABKGroup/RosettaStone

RosettaStone converts Bookshelf files into LEF (Library Exchange Format) and DEF (Design Exchange Format).

7. Loading LEF/DEF into OpenROAD

OpenROAD repository:
https://github.com/The-OpenROAD-Project/OpenROAD

OpenROAD is an open-source physical design tool used for placement, routing, timing analysis, and other stages of chip design.

8. Standard Cell Placement in OpenROAD

After loading LEF and DEF files into OpenROAD, the following steps are typically performed:

1. Read LEF file
2. Read DEF file
3. Perform global placement
4. Perform legalization
5. Run detailed placement

After this stage, both macros and standard cells will be properly placed in the layout.

9. Result and Evaluation

Students should present the final results by comparing the **baseline placement** with the **optimized placement** generated by their model.

1. Placement Visualization

Students must include placement images showing:

- **Baseline Macro + Standard Cell Placement**
- **Optimized Macro + Standard Cell Placement**

These images help visualize how the optimized macro placement affects the overall layout and standard cell distribution.

2. Evaluation Metrics

The following metrics should be reported for both baseline and optimized placements:

- **HPWL (Half-Perimeter Wirelength)**
  Measures the estimated total wirelength of the design.
- **Routing Congestion Estimation**
  Students should present:
  - **RUDY congestion values**
  - **Congestion heatmap visualization**
- **Legalization**
  Verify that the placement is physically valid:
  - No **macro overlap**
  - **Standard cells successfully legalized** within placement rows.

These metrics should be compared between the **baseline and optimized designs** to evaluate improvements in placement quality.